PACKAGE 11 / VOLUME 10

Lab workbook

From prediction to reviewable evidence

20 practical labs · 5 module projects · 10 transfer problems

Contracts, implementation steps, adverse cases and acceptance conditions for every numbered lesson.

September 28, 2026
Graduate self-study specialization

Contents

Lab 46.1 — Cache coherence, memory consistency and synchronization 5

Lab 46.2 — Out-of-order execution, speculation and precise state 6

Lab 46.3 — Architecture simulation, traces and model validation 7

Lab 46.4 — Memory systems, chiplets and multi-objective design 8

Lab 47.1 — CMOS logic, transistor sizing and delay models 9

Lab 47.2 — Static timing analysis, clocks and timing exceptions 10

Lab 47.3 — Synthesis, placement, clock trees, routing and physical checks 11

Lab 47.4 — Power, area, process corners and design-space closure 12

Lab 48.1 — Formal properties, abstraction and assume-guarantee reasoning 13

Lab 48.2 — Clock/reset crossings and asynchronous FIFO verification 14

Lab 48.3 — Fault models, ECC, testability and recovery 15

Lab 48.4 — Hardware security, thermal limits and assurance review 16

Lab 49.1 — Question selection, literature map and thesis scope 17

Lab 49.2 — Proposal review, baselines and risk-reduction milestones 18

Lab 49.3 — Implementation, ablations and controlled evaluation 19

Lab 49.4 — Thesis writing, reproducibility and contribution boundaries 20

Lab 50.1 — Oral defense and independent artifact reproduction 21

Lab 50.2 — Mature-codebase integration, review and maintainable contributions 22

Lab 50.3 — Portfolio, technical interviews and role-specific evidence 23

Lab 50.4 — Long-term learning, maintenance and final handoff 24

Module 46 project — Architecture study with a visible fidelity boundary . . . 25

Module 47 project — Verified RTL through an educational physical flow . . . 26

Module 48 project — Assurance dossier for a data-moving component . . . 27

Module 49 project — One primary thesis, from reviewed question to evidence . . . 28

Module 50 project — Gate J: defend, reproduce, integrate and hand off . . . 29

Additional transfer problems . . . 30

46-A — An ordering claim from a trace . . . 30

46-B — A cache experiment that changes two things . . . 30

47-A — Repairing one kind of slack . . . 30

47-B — Energy versus feasibility . . . 30

48-A — A proof with no traffic . . . 30

48-B — Reset at the worst time . . . 30

49-A — The wrong resampling unit . . . 30

49-B — A favorable metric switch . . . 30

50-A — A clean-room mismatch . . . 31

50-B — The honest portfolio statement . . . 31

Package 11 · Modules 46–50 · Twenty labs and five integrated projects

Work before reading the solution

Create one evidence directory per lesson, such as `work/46.1/`. Separate predictions, changes, actual observations and explanations. Retain commands and raw logs.

Each “Manifest assignment” preserves the original requirements. Reference code does not complete the additional physical-flow, verification, review or thesis work.

Common ten-point lab rubric

Criterion	Points	What the reviewer checks
Contract and prediction	2	Inputs, assumptions, expected result and exclusions are stated before the run.
Implementation or substantive analysis	3	Required transitions, derivations, experiment or artifact exist and match the assignment.
Correctness, boundaries and failure evidence	3	Relevant tests, adverse cases and at least one explained failure are retained.
Evidence and explanation	2	Another reader can follow the commands and distinguish results from assumptions.

Acceptance requires eight points **and all original mandatory conditions**. Missing verification cannot be replaced by writing points. For thesis labs, inspect actual reviewed artifacts and reproduction. Use `templates/LAB_RECORD.md`.

Reference-code map

Lessons	Starting files	Limits
46.1–46.4	`coherence.py`, `scheduler.py`, `cache.py`, `models.py`, C++ publication	Atomic or bounded models, not a production CPU or measured game engine.
47.1–47.4	`models.py`, `code/rtl/`	Arithmetic is tested; consult validation status for optional tool runs.
48.1–48.4	`protocol.py`, `ecc.py`, `security.py`, optional FIFO properties	Finite digital models; no analog reliability or real DMA operation.
49.1–50.4	`analysis.py`, thesis and review templates	Synthetic statistical fixture and workflow aids; no completed learner thesis.

The Python file names above are under `code/reference/`. Run from the package root using the commands in START_HERE. Before editing, copy a reference into your work area or use the dedicated student exercises.

Lab 46.1

Cache coherence, memory consistency and synchronization

Learning outcome: Separate agreement about a location from ordering across locations.

Manifest assignment

Model a small coherence protocol and evaluate message-passing litmus tests under stated language and hardware models. Pass: ownership invariants hold, transient races are considered and coherence is not confused with data-race freedom.

Procedure

Step 1. Run the atomic MSI reference and manually trace the six-operation sequence in the lesson. Label which reads force an owner downgrade.

Step 2. Implement one missing transition in a copy, then enumerate read/write sequences of length up to six over two cores and values zero/one. Check ownership and data invariants after every transition.

Step 3. Draw two invalidation-acknowledgment races. Add transient states to your own model, or provide a reviewed transition table with a retry/serialization policy; the supplied atomic model does not discharge this requirement.

Step 4. Build the C++ publication example. Explain every happens-before edge. Construct a separate all-atomic relaxed litmus description and compare SC versus one named ISA model.

Step 5. Mutate the invalidation operation so that a stale shared copy survives. Retain a minimal failing trace and repair the implementation.

Submission and review

Submit transition table, invariant definitions, exhaustive-trace log, transient-race analysis, C++ build record and two annotated litmus tests.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add a third core and compare bus-broadcast with a directory sharer set. Preserve the same observable single-copy behavior.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 46.2

Out-of-order execution, speculation and precise state

Learning outcome: Increase instruction overlap without changing architectural outcomes.

Manifest assignment

Use a simulator or detailed scheduling model to compare in-order and out-of-order execution on independent and dependent instruction streams. Pass: architectural results agree and bottleneck explanations include true dependencies and resource limits.

Procedure

Step 1. Hand-schedule the four instructions using the stated within-cycle ordering. Run both policies and compare every issue/finish/retirement timestamp.

Step 2. Create three instruction streams: a dependency chain, independent work, and a mixed entity-update DAG. Run windows of one, four and eight with fixed issue width.

Step 3. Compare complete committed value maps for all non-faulting runs. Add malformed dependency and zero-latency inputs and require explicit rejection.

Step 4. Inject an exception at the second instruction and show a younger completed operation excluded from architectural state.

Step 5. Write a bottleneck explanation for one improvement, one non-improvement and one workload not represented by the model.

Submission and review

Submit three DAGs, cycle tables, equivalence tests, fault trace and a one-page model-boundary statement.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add a finite multiplier resource and a separate commit-width parameter. Predict and test a workload whose bottleneck changes.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 46.3

Architecture simulation, traces and model validation

Learning outcome: Use simulation as evidence with a stated fidelity boundary.

Manifest assignment

Run a cache, scheduling or accelerator study with at least one validation case and a sensitivity sweep. Pass: raw configurations reproduce results, modeled versus measured quantities are labeled and conclusions do not exceed the tested fidelity.

Procedure

Step 1. Calculate the six-address worked trace by hand and verify hit/miss order, not just the final count.

Step 2. Implement LRU hit promotion and eviction in the student cache. Test one set, multiple sets, a zero-length trace and invalid geometry.

Step 3. Generate stream, localized and conflict workloads. Compare at least three equal-capacity geometries, then separately sweep capacity.

Step 4. Change additional miss penalty over a stated range and report whether the preferred design changes. Label all derived times as modeled.

Step 5. Write a validation plan with one independent oracle and a held-out case. A hand-derived oracle is acceptable for algorithm verification; it is not hardware validation.

Submission and review

Submit trace files, cache configurations, raw counts, regenerated CSV, oracle comparison, sensitivity results and validation limitations.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add a warmup region and compare cold-start versus steady-state conclusions without discarding adverse startup behavior.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 46.4

Memory systems, chiplets and multi-objective design

Learning outcome: Choose an architecture across performance, power, cost and complexity.

Manifest assignment

Construct a Pareto comparison of at least three architecture configurations. Pass: objectives and assumptions are explicit, dominated designs are identified correctly and a recommendation changes appropriately when a hard constraint changes.

Procedure

Step 1. Implement strict Pareto dominance for all-minimization vectors and test duplicates, ties, empty input and invalid numbers.

Step 2. Reproduce the four-candidate example. Apply hard constraints before selecting a minimum-latency candidate.

Step 3. Build three or more architecture configurations using your cache/scheduling study. Clearly distinguish measured, modeled and assumed objectives.

Step 4. Add a memory-placement or cross-die communication term with declared units. Sweep at least one communication assumption.

Step 5. Change a hard limit and a workload weighting rule, then write the resulting recommendation and its reversal conditions.

Submission and review

Submit objective definitions, frontier table, feasibility filters, evidence-tier labels, sensitivity sweep and a two-page decision record.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Propagate scenario-level uncertainty through selection and report how often a candidate is chosen without labeling that frequency a universal probability of superiority.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 47.1

CMOS logic, transistor sizing and delay models

Learning outcome: Connect gate-level choices to electrical load and timing.

Manifest assignment

Analyze and simulate an inverter chain or compare library-cell drive strengths under several loads. Pass: assumptions are stated, input loading is included and model predictions are compared with available characterized or simulated results.

Procedure

Step 1. Implement the equal-effort chain calculation with positive-load and stage-count validation. Reproduce H=64 for one through six stages.

Step 2. Add an interface polarity requirement and reject functionally inappropriate stage counts before optimizing delay.

Step 3. Analyze how a fixed external driver sees the first-stage capacitance. Compare a larger first stage with an extra internal stage.

Step 4. Use an available educational Liberty library or circuit simulator to compare drive strengths at three loads and two slews; capture exact files and commands. When unavailable, mark this external comparison pending rather than filling in estimates as measurements.

Step 5. Compare normalized predictions with the characterized/simulated results and identify a mismatch with a plausible physical cause.

Submission and review

Submit derivation, normalized sweep, polarity filter, library/simulator identity, characterized comparison and a boundary statement.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Include a wire capacitance term between stages and test whether equal taper remains optimal.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 47.2

Static timing analysis, clocks and timing exceptions

Learning outcome: Check every required path under the correct timing contract.

Manifest assignment

Constrain a small sequential block and inspect worst setup and hold paths. Pass: no unexplained unconstrained endpoints, every exception is justified and a deliberately tightened clock produces an understood violation.

Procedure

Step 1. Reproduce both worked slacks in the timing model and test the stated skew signs with a positive and a negative skew.

Step 2. Read the sample FIFO SDC. Inventory clocks and all input/output ports, including the synchronous reset.

Step 3. In an educational STA flow, load your verified sequential block and inspect worst max/setup and min/hold paths. Save the exact library, constraint and netlist identities.

Step 4. Tighten the period by 20% and explain the changed report. Confirm that a hold violation is not automatically fixed by relaxing the period.

Step 5. Deliberately remove one interface constraint in a copy, detect the uncovered endpoint, and restore it. Provide written justification for every retained exception.

Submission and review

Submit hand calculations, SDC, endpoint coverage inventory, worst-path reports, tightened-clock report and an exception-review record.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Analyze a generated clock with explicit launch/capture edges and compare hand reasoning with the selected timing tool.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 47.3

Synthesis, placement, clock trees, routing and physical checks

Learning outcome: Understand how a logical netlist becomes a physical layout.

Manifest assignment

Run a small block through an educational OpenROAD-compatible flow or inspect a provided open example. Pass: capture stage reports, identify a routing/timing change and state which physical checks were and were not performed.

Procedure

Step 1. Run the FIFO testbench and record the acceptance contract, reset behavior and transaction-order checks.

Step 2. Install or use an already approved educational OpenROAD-compatible flow and pin its revision. Run its official small example before adding the teaching FIFO.

Step 3. Use the supplied config/SDC as a starting point, verifying variable names against your pinned flow. Save synthesis, placement, clock-tree and routing reports.

Step 4. Follow one endpoint pair across stages and identify a cell-delay or wire-delay change. Do not infer this solely from worst-slack differences.

Step 5. Create the performed/not-performed check matrix for DRC, LVS, equivalence, timing, antenna and power integrity. Record every unsupported claim as an exclusion.

Submission and review

Submit RTL/test identities, version-lock record, stage reports, path comparison, final artifact inventory and physical-check coverage matrix.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Compare two placement densities at fixed RTL and constraints; report timing, congestion, cell area and run-to-run sensitivity.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 47.4

Power, area, process corners and design-space closure

Learning outcome: Report PPA with enough assumptions to make comparison meaningful.

Manifest assignment

Compare two verified RTL implementations using identical flow assumptions. Pass: report area, worst timing and activity-based power estimates where supported, with uncertainty and no unsupported commercial-silicon claim.

Procedure

Step 1. Implement the switching-power and energy-per-item calculations with unit-aware inputs and rejection of negative or nonfinite values.

Step 2. Choose two functionally equivalent FIFO/controller implementations or a reviewed pair of small RTL blocks. State equal useful-work and latency requirements.

Step 3. Run both through the same educational flow, corners and constraints. Collect cell area, setup/hold results and power estimates where supported.

Step 4. Use representative and high-activity stimuli, recording activity coverage and default assumptions. Leave unsupported power fields unknown.

Step 5. Apply a hard constraint and show the feasible frontier. Write a final claim that includes both results and missing signoff coverage.

Submission and review

Submit equivalence/transaction tests, PPA table with units, activity records, matched flow identities, feasible-design decision and assurance boundary.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Evaluate energy over a burst-plus-idle interval and determine when the faster design can lose after transition overheads are included.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 48.1

Formal properties, abstraction and assume-guarantee reasoning

Learning outcome: Prove useful properties without hiding failures in assumptions.

Manifest assignment

Prove a queue, arbiter or transaction property and inject a violation. Pass: assumptions are reviewed independently, contention is covered and bounded, proven, failed and unknown results are reported separately.

Procedure

Step 1. Write the conservative FIFO transition relation and prove occupancy bounds on paper by cases.

Step 2. Run finite-state exploration for depth-two storage with a two-value alphabet. Record whether exploration reaches a fixed point, rather than labeling a depth-limited sample exhaustive.

Step 3. Run the RTL simulation scoreboard and inspect the optional formal accounting/flag properties. Run the SBY proof/cover tasks when the required tools are available; report each actual status separately.

Step 4. Inject a full-write occupancy bug and a data-order bug separately. Explain which properties or tests detect each.

Step 5. Create a requirement-to-evidence table and have a reviewer audit assumptions, contention coverage and unresolved obligations.

Submission and review

Submit transition proof, explored-state record, assertion/cover list, bug counterexamples, exact result categories and verification traceability.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add bounded fairness and prove a latency bound for eventual consumption. State exactly which environment guarantee makes the bound true.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 48.2

Clock/reset crossings and asynchronous FIFO verification

Learning outcome: Check digital protocol correctness while acknowledging analog risk.

Manifest assignment

Model an asynchronous FIFO or event handshake with independent clocks and reset sequences. Pass: no loss/duplication under the stated protocol, full/empty behavior is checked and physical synchronizer assumptions are listed separately.

Procedure

Step 1. Hand-trace the two-stage toggle handshake with source/destination periods two and three and at least two initial phase choices.

Step 2. Run period pairs (2,3), (3,2), (5,7) and coincident edges. Add destination backpressure and compare accepted versus delivered sequences after drain.

Step 3. Test mailbox-full behavior and a stopped destination clock. Verify stable payload while busy and distinguish delayed delivery from corruption.

Step 4. Assert coordinated reset with an outstanding item. Record the dropped in-flight item, start a new epoch and verify no old item appears afterward. Reject unilateral reset in this model.

Step 5. Write a physical-assumptions review covering synchronizer structure, metastability, reset release and multi-bit timing. Derive Gray adjacency as a separate FIFO-extension exercise.

Submission and review

Submit edge-by-edge traces, sequence scoreboards, reset-epoch accounting, full/empty-equivalent tests and separate digital/physical assurance statements.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Implement a Gray-pointer asynchronous FIFO with a power-of-two depth, explicit wrap bit and a reviewed full/empty derivation. This is an extension, not a supplied production design.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 48.3

Fault models, ECC, testability and recovery

Learning outcome: Design for detectable failures rather than assuming perfect hardware.

Manifest assignment

Implement a parity and a small single-error-correcting code model, then inject faults. Pass: enumerate the supported correction/detection claims, test multi-bit counterexamples and define a recovery response.

Procedure

Step 1. Implement even parity and the extended Hamming encoder/decoder with the documented bit-position convention.

Step 2. Reproduce payload ten encoding to 0xD2 and verify every single-bit flip returns the original payload.

Step 3. Enumerate all sixteen clean words, 128 single-bit faults and 448 double-bit faults. Require untrusted payload to be absent in double-error results.

Step 4. Enumerate three-bit faults and preserve a specific miscorrection example. Contrast it with an even-parity two-bit miss.

Step 5. Define recovery for a transient asset cache and a persistent save-state path, including retries, escalation, logging and a stopped-processing condition.

Submission and review

Submit encoder/decoder, exhaustive fault log, bit-layout derivation, multi-bit counterexamples and a recovery-policy table.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add interleaving for a stated burst-fault model and analyze the new assumptions rather than merely counting more parity bits.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 48.4

Hardware security, thermal limits and assurance review

Learning outcome: Include isolation and failure containment in the design contract.

Manifest assignment

Write a threat-and-failure review for the accelerator and test malformed descriptors, denied access and reset under load in simulation. Pass: isolation limits, protection assumptions and unresolved risks are explicit; no unauthorized target testing occurs.

Procedure

Step 1. Implement the descriptor validator and test exact-end fits, overruns, negative values, zero length, misalignment, wrong owner and stale generation.

Step 2. Add a bounded local work queue. Verify rejected descriptors do not modify its modeled memory or pending count.

Step 3. Queue work, reset, then submit a stale completion and stale descriptor. Require rejection under the new generation.

Step 4. Write a threat/failure table identifying assets, trust boundaries, attacker capabilities, containment mechanisms and exclusions.

Step 5. Calculate the hypothetical thermal example, then create a read-only monitoring/abort plan for any future authorized hardware run. No protection settings are changed in this lab.

Submission and review

Submit rejection tests, reset-under-load trace, memory-effect checks, threat/failure matrix and a bounded assurance statement.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add a two-phase commit protocol for modeled partial writes and demonstrate cancellation or recovery after each interruption point.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 49.1

Question selection, literature map and thesis scope

Learning outcome: Turn a broad interest into a falsifiable, feasible research question.

Manifest assignment

Write a 3-5 page proposal with a research question, 8-15 relevant sources, baseline plan, constraints and exclusions. Pass: a reviewer can explain what observation would weaken or refute the main hypothesis.

Procedure

Step 1. Choose one primary specialization and write three candidate research questions. Reject two with an explicit feasibility or evidence argument.

Step 2. Build an 8–15-source literature map using original papers and official documentation. Record search date, source identity and competing findings.

Step 3. Write a 3–5 page proposal with question, hypothesis, refutation condition, baseline, metric, correctness oracle, constraints and exclusions.

Step 4. Identify a minimal feasibility experiment that can be completed before the main implementation.

Step 5. Ask a qualified reviewer or peer panel to explain the hypothesis back to you and record corrections before approval.

Submission and review

Submit proposal, literature map, rejected alternatives, feasibility plan and dated reviewer decision.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Write a two-paragraph adversarial review arguing why the hypothesis could fail, then revise the evaluation to distinguish those explanations.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 49.2

Proposal review, baselines and risk-reduction milestones

Learning outcome: Make failure cheap before the expensive implementation phase.

Manifest assignment

Deliver a running baseline, risk register and milestone plan. Pass: the primary metric can be measured, a small feasibility experiment exists and each major risk has a mitigation or explicit acceptance decision.

Procedure

Step 1. Run the chosen baseline end to end on a minimal workload and check its output against a trusted oracle or documented tolerance.

Step 2. Complete the experiment-contract template, including the independent sampling unit and handling of failed runs.

Step 3. Estimate runtime and variance with a small pilot. Recalculate the complete run matrix and resource requirements.

Step 4. Create a risk register with at least five project-specific risks, each with a trigger and mitigation/acceptance decision.

Step 5. Hold a baseline/milestone review and record a continue/narrow/revise decision before major implementation.

Submission and review

Submit baseline artifact, environment record, oracle check, pilot data, experiment contract, resource budget, risk register and review decision.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Compare two plausible baseline choices and explain how each could bias the apparent contribution.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 49.3

Implementation, ablations and controlled evaluation

Learning outcome: Execute the study without changing the question to fit a favorable result.

Manifest assignment

Run the approved evaluation with baselines, at least one ablation and one sensitivity study. Pass: scripts regenerate results, failed cases remain visible and claims use the preregistered primary metric or clearly disclose deviations.

Procedure

Step 1. Run baseline, complete candidate and at least one justified ablation on the approved workload set, with correctness checks before timing summaries.

Step 2. Run one sensitivity sweep that could plausibly reverse the conclusion. Retain adverse cases, failures and timeouts.

Step 3. Use the supplied synthetic pairs to test the analysis code, then analyze real study data separately with explicit provenance.

Step 4. Compute the primary metric and justified uncertainty analysis at the prespecified independent unit. Record seed, quantile definition and replication count for resampling.

Step 5. Regenerate the central table from a clean output directory and write a deviations/failure report without changing the original protocol file.

Submission and review

Submit raw data, status/failure ledger, run manifests, ablation table, sensitivity results, analysis scripts and protocol amendments.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Add a paired permutation or hierarchical analysis justified for the study design and compare the interpretation with the basic bootstrap.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 49.4

Thesis writing, reproducibility and contribution boundaries

Learning outcome: Write a technical argument another engineer can audit.

Manifest assignment

Produce a thesis-style report, typically 30-60 pages plus appendices as a planning guide, and a complete artifact package. Pass: a reviewer traces five major claims to evidence and reproduces the central table or experiment.

Procedure

Step 1. Write the thesis-style report with question, related work, method, correctness, evaluation, limitations and contribution boundaries.

Step 2. Create a ledger for at least five major claims and have a reviewer trace them to raw inputs and scripts.

Step 3. Package a clean artifact with short and full reproduction commands, dependency lock, provenance, expected output and rights/assistance disclosure.

Step 4. Run the artifact from a clean directory or isolated environment and test file-integrity validation.

Step 5. Incorporate reviewer corrections, regenerate every final figure/table and tag the revised release.

Submission and review

Submit thesis report, technical appendices, claim ledger, reproducible artifact, integrity manifest, provenance/rights record and revision log.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Ask a second reviewer unfamiliar with the project to find and reproduce a secondary result using only the entry document.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 50.1

Oral defense and independent artifact reproduction

Learning outcome: Show that the learner owns the reasoning and implementation.

Manifest assignment

Hold a 30-45 minute technical defense with a qualified reviewer or peer panel and a separate artifact run. Pass: the central result is reproduced, major questions receive evidence-based answers and required revisions are completed.

Procedure

Step 1. Prepare the 30–45 minute defense and distribute the release, abstract, claim ledger and reviewer instructions in advance.

Step 2. Include one central derivation, one failed/adverse case and one live implementation or contract question.

Step 3. Have a reviewer independently execute the central-result procedure, recording their own environment and deviations.

Step 4. Score the Gate J rubric, record required revisions and distinguish blocking issues from optional extensions.

Step 5. Complete revisions, rerun affected checks and obtain a dated closure decision from the reviewer.

Submission and review

Submit defense outline, reviewer questions, independent reproduction log, rubric, revision responses and closure record.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Ask the reviewer to choose an unseen workload or small parameter change and predict the outcome before running it.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 50.2

Mature-codebase integration, review and maintainable contributions

Learning outcome: Transfer a prototype into an engineering environment with other owners.

Manifest assignment

Integrate one tested improvement into a mature open codebase or a maintained local fork. Pass: a focused patch, regression test, design rationale and reviewer response are included, whether or not an upstream project accepts it.

Procedure

Step 1. Choose an authorized mature project or maintained fork, pin its base revision and read its contribution/build guidance.

Step 2. Write a one-page integration proposal identifying the narrow behavior change, compatibility effects and out-of-scope work.

Step 3. Implement a focused patch with a regression/equivalence test and run the relevant clean-build checks.

Step 4. Obtain review from another person and respond to each substantive comment with evidence and commit references.

Step 5. Produce a patch series or local release with rollback instructions, ownership, known issues and an honest upstream status.

Submission and review

Submit base/patch identities, focused diff, tests, benchmark evidence where relevant, review discussion and integration/handoff notes.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Backport the patch to one supported branch and document compatibility differences without duplicating the implementation unnecessarily.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer's decision. Do not replace this record with the answer key.

Lab 50.3

Portfolio, technical interviews and role-specific evidence

Learning outcome: Present concrete engineering competence without inflated claims.

Manifest assignment

Prepare three portfolio case studies and complete one systems, one algorithm and one specialization mock interview. Pass: every claimed result is supported by an artifact and assistance or third-party contributions are clearly attributed.

Procedure

Step 1. Prepare three case studies from existing systems, algorithm/numerical and thesis artifacts; do not create unnecessary extra capstones.

Step 2. Add evidence paths, role relevance, exact personal contribution, assistance disclosure and a limitation to each case study.

Step 3. Complete one systems, one algorithm and one specialization mock interview with another person or a clearly labeled practice partner.

Step 4. Record scoring, hints and missed concepts, then solve one unseen transfer problem for the primary gap.

Step 5. Perform a rights/privacy/claim audit before publishing or sharing the portfolio.

Submission and review

Submit three case studies, three interview records, scoring/reflection, one transfer solution and a publication-rights checklist.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Rewrite one case study for two different role audiences without changing the facts or inflating scope.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Lab 50.4

Long-term learning, maintenance and final handoff

Learning outcome: Keep the knowledge and artifacts useful after the course ends.

Manifest assignment

Complete Gate J: archive the thesis artifact, produce a maintainer handoff and write a 90-day learning plan with one primary gap and one real contribution goal. Pass: another person can locate, build and understand the project without the author's machine.

Procedure

Step 1. Assemble the final thesis release and verify its complete inventory, integrity manifest, provenance and rights/assistance record.

Step 2. Write the maintainer runbook with architecture, contracts, commands, known issues, troubleshooting, ownership and rollback.

Step 3. Have another person locate, build, test and explain the artifact from the entry document. Record and repair every blocking documentation gap.

Step 4. Complete a 30/60/90-day plan centered on one primary gap and one maintained contribution, with measurable review points.

Step 5. Close the Gate J checklist only after the independent reproduction, defense and blocking revisions are complete; record remaining nonblocking limitations honestly.

Submission and review

Submit final release, handoff runbook, independent handoff log, 90-day plan and signed/dated educational completion record.

Apply the ten-point rubric. The original pass conditions above are mandatory; identify the exact file or reviewed argument that satisfies each one. Record tool outcomes as passed, failed, inconclusive or not run.

Extension after the core lab

Run the archived central experiment after a controlled dependency update and publish a compatibility/correction note without altering the historical release.

Lab record prompts

Write your predicted result, the hardest boundary case, one deliberate failure and its explanation, evidence paths, remaining limitations and the reviewer’s decision. Do not replace this record with the answer key.

Module 46 project

Architecture study with a visible fidelity boundary

Original brief

Perform an architecture study using a pinned simulator or a carefully bounded analytical model. Compare at least three configurations on representative and adverse game-related workloads, and validate the model against a known reference where possible.

Scope and integration

Select a game-related operation such as entity-component iteration, visibility-list traversal or a command-queue access pattern. Use at least one representative workload and one adverse workload. Begin with the supplied three equal-capacity cache geometries only as a mechanics check; then justify an actual workload source or a synthetic generator tied to an explicit mechanism. Compare at least three configurations in a pinned simulator or a bounded analytical model. A trace cannot support hypotheses about execution-dependent address generation unless the limitation is addressed.

Required evidence package

Submit the model contract, trace provenance and hashes, validation tests against an oracle or hand case, configuration table, metric definition, full result table, at least one adverse result and a sensitivity study. Include a coherence/consistency explanation and the precise-state exercise as separate prerequisite evidence. A table of three unvalidated synthetic timings is not sufficient.

Acceptance and stopping conditions

Acceptance requires equal comparison scope, explicit changed dimensions, correctness before speed, at least one known-reference validation and claims bounded to the model. Negative results are admissible. Gate failure triggers include calling modeled cycles measured FPS, silently omitting adverse workloads, or attributing all effects to associativity after changing capacity.

Review sequence

First review the contract and source provenance. Next reproduce a minimal correct case and an adverse case. Inspect every central claim against its evidence. Record discrepancies before discussing extensions. Agree on a repair or a narrower defensible claim, and then rerun the affected acceptance checks. A reviewer’s unanswered question is an open issue, not an automatic rejection or approval.

Module 47 project

Verified RTL through an educational physical flow

Original brief

Carry a verified small RTL block through an educational synthesis and physical-design flow. Report constraints, timing, area and power-estimation assumptions. The result is a learning artifact, not fabrication-ready commercial signoff.

Scope and integration

Use the fixed-depth synchronous FIFO or an equally small verified block. Freeze its interface and reset semantics before synthesis. Establish simulation evidence and a property plan, then use an authorized educational standard-cell library with pinned synthesis and physical-design tools. Record constraints at each stage and keep intermediate reports; a final screenshot alone cannot reconstruct the analysis.

Required evidence package

Submit RTL and test versions, reset/handshake contract, SDC with time and load units, tool/library identity, synthesis and implementation logs, stage-specific warnings, setup/hold reports, unconstrained-path review, area definition, power-estimation basis and an alternatives comparison. Inspect a completed open example only where the lesson explicitly permits it; the integrated project requires your own small block’s flow evidence.

Acceptance and stopping conditions

Acceptance requires meaningful constraints on required paths, timing and area from actual reports rather than the Python calculator, correct labeling of pre-route versus post-route estimates, and a review of unresolved physical checks. Tools not executed remain not run. This project is not fabrication approval; no silicon purchase or protected-process access is implied.

Review sequence

First review the contract and source provenance. Next reproduce a minimal correct case and an adverse case. Inspect every central claim against its evidence. Record discrepancies before discussing extensions. Agree on a repair or a narrower defensible claim, and then rerun the affected acceptance checks. A reviewer’s unanswered question is an open issue, not an automatic rejection or approval.

Module 48 project

Assurance dossier for a data-moving component

Original brief

Create a verification dossier for the hardware capstone: requirements-to-tests traceability, assertions, coverage, clock/reset review, fault injection and a clearly bounded assurance claim.

Scope and integration

Reuse the FIFO or select a small queue/DMA-like controller with an explicitly bounded trust model. Pair properties with reachable traffic, model CDC and reset or justify a single-clock boundary, inject stated fault classes, and evaluate invalid descriptors and recovery. Keep the digital/analog boundary visible. A symbolic proof of a count invariant alone does not establish payload integrity.

Required evidence package

Submit a property/assumption/cover matrix, solver outcomes and counterexamples, simulation scoreboard results, reset-epoch contract, CDC review, fault campaign counts and response policy, malformed-input tests, residual risks and evidence for at least one deliberate mutation. State which conditions are proved, tested, inspected, assumed or unaddressed.

Acceptance and stopping conditions

Acceptance requires no unexplained contradiction between the contract and evidence, explicit handling of detected-uncorrectable errors, no stale completion accepted after modeled reset, and an honest reliability limit. Passing a finite protocol simulator does not establish MTBF; SECDED does not promise correction of arbitrary multi-bit faults. Unilateral reset cannot be waved away as an ordinary empty queue.

Review sequence

First review the contract and source provenance. Next reproduce a minimal correct case and an adverse case. Inspect every central claim against its evidence. Record discrepancies before discussing extensions. Agree on a repair or a narrower defensible claim, and then rerun the affected acceptance checks. A reviewer’s unanswered question is an open issue, not an automatic rejection or approval.

Module 49 project

One primary thesis, from reviewed question to evidence

Original brief

Prepare and execute a thesis-scale project in one primary specialization. Budget 400-800 focused hours as a planning range, with scope adjusted through review. The deliverable is a defensible contribution and reproducible artifact, not a promise of publication.

Scope and integration

Choose one specialization, not a collection of unrelated systems. Produce the 3–5-page proposal and literature map, obtain a technical scope review, execute a running baseline and feasibility experiment, then freeze the primary comparison protocol. Implement the candidate and a reference path, complete at least one ablation and one sensitivity study, and write the thesis-style argument.

Required evidence package

Deliver proposal and review actions, 8–15 relevant starting sources with claim mapping, baseline record, risk register, versioned design decisions, tests, raw data, analysis scripts, deviations, negative cases and a claim-evidence ledger. The report should typically fit 30–60 pages plus appendices, but quality and traceability determine acceptance. Refer to the Gate J guide for the release structure.

Acceptance and stopping conditions

Acceptance requires a feasible falsifiable question, an actually running baseline, a controlled comparison and a report that distinguishes new work, reproduced work, borrowed code and unsupported generalization. The supplied reference outputs cannot be passed off as your experiment. Gate J's independent reproduction and defense occur in Module 50 and reuse this same thesis.

Review sequence

First review the contract and source provenance. Next reproduce a minimal correct case and an adverse case. Inspect every central claim against its evidence. Record discrepancies before discussing extensions. Agree on a repair or a narrower defensible claim, and then rerun the affected acceptance checks. A reviewer's unanswered question is an open issue, not an automatic rejection or approval.

Module 50 project

Gate J: defend, reproduce, integrate and hand off

Original brief

Gate J: defend the thesis, pass an independent artifact reproduction, complete a maintainable handoff and publish an honest portfolio where rights permit. This completes the self-study program, not an accredited degree.

Scope and integration

Freeze a release candidate from Module 49. A technically capable reviewer should reproduce the central table or result from a clean workspace, challenge a derivation and a failure case, and inspect a live implementation question during the 30–45-minute defense. Address review actions and prepare a maintainer handoff. Separately demonstrate a reviewable mature-codebase integration and an honest role-specific portfolio.

Required evidence package

Deliver a signed or explicitly attributed reviewer record, reproduced outputs and discrepancy log, defense questions and revisions, final hashes and release notes, integration patch with base/changed tests, three portfolio case studies, three mock-interview records and a single-priority 90-day continuation plan. Prior curriculum projects may supply the extra case studies; three new thesis projects are not required.

Acceptance and stopping conditions

Gate J closes only when the central result is independently reproduced or the claim is explicitly narrowed and independently verified, all critical review conditions are satisfied, and the learner can defend both a derivation and a failed case. A weighted score cannot erase a safety, integrity or central-correctness blocker. This completes a self-study route, not a degree.

Review sequence

First review the contract and source provenance. Next reproduce a minimal correct case and an adverse case. Inspect every central claim against its evidence. Record discrepancies before discussing extensions. Agree on a repair or a narrower defensible claim, and then rerun the affected acceptance checks. A reviewer’s unanswered question is an open issue, not an automatic rejection or approval.

Additional transfer problems

These ten problems are additional practice, separate from the sixty lesson quiz questions. Attempt them before opening the transfer-solution appendix. State units and assumptions where applicable.

46-A — An ordering claim from a trace

A worker reports that a million relaxed all-atomic store-buffering runs never produced zero/zero. Write the strongest justified statement, the unjustified stronger statement, and one evidence source that could decide whether the outcome is allowed.

46-B — A cache experiment that changes two things

Compare a cache with 8 sets × 1 way × 64 bytes and one with 8 sets × 2 ways × 64 bytes. What changed? Construct a same-capacity associativity comparison, and state one limitation that survives the repair.

47-A — Repairing one kind of slack

Starting from setup slack +0.25 ns and hold slack −0.06 ns, add a buffer with +0.08 ns minimum delay and +0.12 ns maximum delay. Assume everything else unchanged. What are the new slacks?

47-B — Energy versus feasibility

Candidate P consumes 4 mW and processes 2 million items/s. Q consumes 3 mW and processes 1 million items/s. Which has lower energy per item? Which is feasible under a 3.5 mW hard power ceiling?

48-A — A proof with no traffic

A queue harness assumes push is always false and proves that every popped value was previously pushed. Explain the vacuity risk and specify a cover or mutation that improves confidence.

48-B — Reset at the worst time

In the one-entry mailbox, the source has toggled request and is holding a payload. Only the destination is reset. Explain why treating this as an ordinary empty state is unsafe.

49-A — The wrong resampling unit

A study has twelve independently selected scenes, each producing 10,000 frame times for both configurations. Why is sampling all 120,000 frames as independent potentially misleading? Specify a defensible paired primary analysis.

49-B — A favorable metric switch

The approved primary metric is p95 frame time. After testing, the candidate improves the mean but worsens p95. Write an honest result sentence and one allowed next step.

50-A — A clean-room mismatch

The independent reviewer reproduces the final table except one workload that is 15% slower. The thesis conclusion depends on that row. What should happen before Gate J closes?

50-B — The honest portfolio statement

Rewrite “I built a production GPU and earned a master’s degree” for a learner who implemented a queue accelerator, tested it in an educational flow and completed this self-study route.